

CLOUD-1 — #336: collapse the redundant route-local #323 plumbing, and pin what replaces it

Batch of 2026-08-26 · merge order 1 of 5 (FIRST — touches the contended planner/handlers area; everything local resumes after you land).

```
task_id: gv-336-cloud-1
issue: 336
branch: cloud/336-collapse-route-local
base: main          # 682f3061 or later
adr_number: 0084     # ASSIGNED. 0083 is taken. Do not pick "the next free" one.
sign_commits_as:
  name: Claude_Max
  email: 262510778+tom2025b@users.noreply.github.com
  # per-commit, ALWAYS: git -c user.name=Claude_Max -c
  user.email=262510778+tom2025b@users.noreply.github.com commit ...
  # NEVER a bare `git commit` or bare `git merge`.
allowed_paths:
  - crates/git-vista-server/src/middleware.rs
  - crates/git-vista-server/src/handlers/commit.rs
  - crates/git-vista-server/src/planner/commit_exec.rs
  - crates/git-vista-server/src/**          # wire-level tests may need a test module
home
  - docs/adr/
forbidden_paths:
  - crates/git-vista/src/**                # frontend is another crew's lane today
  - ci/browser/**
deliverables:
  - branch pushed, PR opened with "Closes #336", body per the rules below
  - ADR 0084 + docs/adr/README.md index entry (read 0080 and 0083 first, match the
    house shape)
```

Environment truths — read before your first test run

- ~320 `git-vista-server` tests fail in your container on unmodified `main` (the strict sandbox tier needs `landlock_abi >= 6 + bwrap`; your kernel lacks Landlock). Run the suite on unmodified `main` FIRST, keep that failing set, and compare yours against it. **Only the difference is yours.** State both counts in the PR body. Never report a sandbox refusal as a defect; never claim the suite is green.
- **Build the sandbox helper before any server test run**, or hundreds more fail at spawn for a missing binary, not a real cause: `cargo build -p git-vista-server --bin gv-sandbox`
- **The browser leg cannot run in your container.** Say so in the PR body explicitly: "ci/browser/run.sh unrun — cloud container". The owner runs it on real hardware before merge.

Truth-checked state (verified against main 682f3061, 2026-08-26)

The issue's citations have drifted — the planner was split into modules since:

- main's general fix: `rewrap_error` at `middleware.rs:215`, with `MAX_ERROR_BODY = 64 * 1024` at `middleware.rs:43` and the `to_bytes(response.into_body(), MAX_ERROR_BODY)` sniff at `:220`.
- the route-local layer: `amend_route_response` at `handlers/commit.rs:177`; `amend_refusal` (returns `Response`) at `planner/commit_exec.rs:491`; `amend_refusal_body` at `planner/commit_exec.rs:511`.

Re-verify each before editing — and treat any further drift the same way: open the file, never trust the citation.

The job

323 got fixed twice: main's general `rewrap_error` byte-sniff, and the

`amend_route`'s local relabeling. Both live on main; the local one is redundant for the common case but genuinely stronger in one edge: it sniffs the `String` BEFORE axum caps the body at `MAX_ERROR_BODY`, so a refusal over 64 KiB keeps its content where the general path collapses it to empty (reachable via a hook printing a large rejection).

1. **Decide, and write ADR 0084 saying which and why:** fix the `MAX_ERROR_BODY` ordering in `rewrap_error` so the general mechanism also preserves oversized refusals (preferred if achievable without buffering unbounded bodies — say how you bound it), OR keep the route-local layer and document exactly the edge it exists for. The decision criterion is the owner's standing rule: the thorough, complete mechanism over the quick one; one mechanism that covers everything beats two that each cover most.
2. **If collapsing:** remove `amend_refusal_body` / `amend_route_response`, restore `amend_refusal` to the plain (`StatusCode`, `String`) shape — WITHOUT disturbing the revert-conflict (#327) work that shares these files.
3. **Pin the incidental coverage — but the issue is half wrong here, and knowing which half saves you the work of inventing a pattern that already exists.** The issue claims "no wire-level test covers them: every existing `FetchError`/`PullError` test calls planner functions directly, bypassing the router and `api_contract` entirely." Verified against 405a7644:
 - **/api/pull IS covered.** `the_strategy_mandate_is_a_400_through_a_real_router` (`handlers/pull.rs:360`) builds a real `Router`, layers `middleware::api_contract`, and asserts a 400 refusal whose typed DTO body is parsed directly — its own doc comment states it mirrors `/api/amend-commit`'s contract and notes `/api/fetch` builds `FetchError` the same way.
 - **/api/fetch is NOT covered.** The only `route("/api/fetch", ...)` in the tree is the production registration at `main.rs:575`.

So: extend the existing pull test's shape to `/api/fetch` rather than inventing one, and add whatever the surviving mechanism from step 1 needs to make a narrowing of the general sniff turn something red. If your own check disagrees with either bullet, say so in the PR body and follow your check — not this document. 4. **Mutation-prove whichever mechanism survives, two different ways** — remove the sniff, then weaken it (e.g. relabel only sub-64KiB bodies) — red at different assertions, byte-identical restore verified with diff.

Acceptance

1. ADR 0084 records the decision with the >64 KiB edge stated honestly.
2. The surviving mechanism is mutation-proved two different ways (evidence in the PR body: the two red assertion lines, verbatim).
3. Wire-level `/api/fetch` and `/api/pull` refusal tests exist and fail if the general sniff is narrowed.
4. `cargo fmt --all clean · cargo clippy --all-targets -- -D warnings clean · server suite diff-vs-baseline` is zero new failures.
5. PR body: both baseline counts, the browser-leg-unrun line, and the ADR summary. Sign the PR body with your session tag.

Written by fable · 2026-08-26 · truth-checked against 682f3061 the same morning.